# Projet Génie Logiciel

Projet LambdaSys

Année 2025-2026

**EL MAKHLOUFI Tarik**
**BOURGER Léo**
**JAZIRI Jihène**

Encadré par : **FRANCISCI Dominique**

UFR Sciences & Techniques
Université de Toulon

# Table des matières

**1 Introduction** **2**
1.1 Présentation du projet . . . . . 2
1.2 Approche du sujet . . . . . 2
1.3 Présentation de l'équipe . . . . . 2
1.4 Planning . . . . . 3

**2 Spécifications Logicielles** **5**
2.1 Exigences Fonctionnelles (Métier) . . . . . 5
2.2 Exigences d'Interface et Non-Fonctionnelles . . . . . 5

**3 Conception et Maquettage** **6**
3.1 Maquettage de l'IHM . . . . . 6
3.2 Modélisation UML . . . . . 6
3.2.1 Axe Fonctionnel (Cas d'utilisation) . . . . . 6
3.2.2 Axe Statique (Diagramme de Classes) . . . . . 7
3.2.3 Axe Dynamique (Diagramme de Séquence) . . . . . 7

**4 Implémentation et Bilan Technique** **9**
4.1 Choix techniques : Introspection et Généricité . . . . . 9
4.2 Problèmes rencontrés et solutions . . . . . 9

**5 Difficultées rencontrées** **10**

**6 Conclusion** **11**

# 1. Introduction

## 1.1 Présentation du projet

L'objectif du projet LambdaSys et de créer un logiciel générique permettant de modéliser des systèmes techniques.

Le logiciel doit être capable de pouvoir générer des jeux de données pour ces systèmes et doit pouvoir également gérer les anomalies dans le but de pouvoir entrainer des modèles d'IA discriminantes ou génératives.

## 1.2 Approche du sujet

Pour répondre aux exigences du sujet, il est nécéssaire de structurer le projet dès le départ afin de construire de manière optimal le logiciel.

Il est absolument nécéssaire de concerver l'idée générative du logiciel donc dans un premier temps il faut se concentrer sur les exigences.

Tout d'abord il faut concevoir une petite maquette pour avoir une première approche du logiciel afin de pouvoir établir les différentes exigences logicielles. Après avoir clairement définis ces exigences il faut concevoir l'axe fonctionnelle en commençant par le diagramme de cas d'utilisation.

Une fois le diagramme conçu, il faut faire un diagrammde séquence qui répond aux exigences du logiciel.

Ensuite vient l'axe statique avec le diagramme de classe qui est l'étape cruciale qui nous servira de base pour la parti codage du logiciel.

Une fois que tout l'UML est finalisé, on pourra commencer la construction du logiciel qui sera réalisé en Python Orienté Objet.

Enfin pour avoir un suivi complet du projet grâce à Github.

## 1.3 Présentation de l'équipe

Ce projet sera réalisé par 3 collaborateurs :

- Bourger Léo

- El makhloufi Tarik

- Jaziri Jihène

Il est important de répartir de façon équitable les différentes tâche du projet. Donc il n'y a pas de rôle défini pour chaque collaborateur.

Pour chaque séance de projet on définit clairement les tâches du jour et on les répartie entre chaque personne afin d'avoir un suivi parallèle et au fur et à mesure de la séance, on réalise une mise en commun pour un suivi plus fluide .

## 1.4 Planning

Voici le planning complet du projet représentant les tâches de chque personne durant les séances de TP :

| Date | Membre(s) | Tâche réalisée |
|---|---|---|
| 02/02/2026 | Toute l'équipe | Construction de l'équipe |
| 03/02/2026 | Toute l'équipe | Analyse du cahier des charges et Conception des Exigences |
| 10/02/2026 | Léo | Finalisation des exigences et début de l'Axe Fonctionnelle |
| 10/02/2026 | Tarik | Création du dépot Git et début de l'Axe Fonctionnelle |
| 10/02/2026 | Jihène | Réalisation de la maquette du logiciel et début de l'Axe Fonctionnelle |
| 17/02/2026 | Toute l'équipe | Modélisation UML et Avancement de l'axe Fonctionnelle |
| 03/03/2026 | Tarik | Mise en commmun de l'Axe Fonctionnelle et début de l'Axe Statique |
| 03/03/2026 | Léo | début de l'Axe Statique |
| 03/03/2026 | Jihène | Création de la première fenêtre Python |
| 10/03/2026 | Toute l'équipe | Avancement de l'Axe Statique et début de l'Axe Dynamique |
| 17/03/2026 | Jihène | Finalisation de l'axe Statique et Dynamique |
| 17/03/2026 | Léo | Finalisation de l'axe Statique et Dynamique |
| 17/03/2026 | Tarik | Finalisation de l'axe Statique et Début de l'implémentation en Python |
| 24/03/2026 | Toute l'équipe | Avancement du logiciel en python |
| 31/03/2026 | Toute l'équipe | Avancement du logiciel en python |
| 07/04/2026 | Tarik | Avancement du Compte rendu |
| 07/04/2026 | Jihène | Création du PowerPoint |
| 07/04/2026 | Léo | Mise à jour de l'UML en fonction de l'implementation en Python (Diagramme de classe) |
| 14/04/2026 | Toute l'équipe | Finalisation de l'implémentation Python |

Table 1.1 – Extrait du carnet de bord du projet LambdaSys

# 2. Spécifications Logicielles

Afin de garantir la généricité et la robustesse du logiciel LambdaSys, nous avons établi un cahier des charges précis, divisé en trois catégories d'exigences.

## 2.1 Exigences Fonctionnelles (Métier)

- **MODELISATION :** Le logiciel doit permettre d'importer des modèles existants via des fichiers externes contenant les lois physiques du système. L'utilisateur doit pouvoir visualiser l'arborescence des composants de ce système.
- **CAPTEUR :** Chaque composant peut disposer de capteurs virtuels configurables (Type de mesure, Fréquence d'échantillonnage, Unité).
- **SIMULATION :** Le logiciel doit permettre de définir la durée totale de la simulation et le pas de temps, et de générer les valeurs nominales à chaque itération.
- **ANOMALIE :** Le système doit permettre d'injecter des défauts maîtrisés sur les composants pour dégrader leur comportement de manière réaliste.
- **EXPORT :** Les données issues de la simulation doivent être exportables au format CSV pour permettre l'entraînement de modèles d'IA.

## 2.2 Exigences d'Interface et Non-Fonctionnelles

- **UI :** L'interface doit être ergonomique, utilisant des panneaux ancrables (docks) pour séparer l'explorateur de système, les propriétés et la visualisation des résultats.
- **TECH :** Le développement doit être réalisé en Python Orienté Objet (POO) en respectant l'architecture MVC (Modèle-Vue-Contrôleur). Le projet doit être versionné sous Git.

# 3. Conception et Maquettage

## 3.1 Maquettage de l'IHM

L'interface graphique a été conçue pour s'apparenter à un environnement de développement ou de simulation industriel. Elle est divisée en trois zones principales :

1. **Un explorateur de système (à gauche) :** Affiche l'arborescence des composants et de leurs capteurs respectifs suite à l'importation d'un modèle.
2. **Un panneau de propriétés (à droite) :** Permet à l'utilisateur de configurer dynamiquement le composant ou le capteur sélectionné (modification de la fréquence, injection d'anomalie).
3. **Une zone centrale :** Destinée à afficher le journal d'exécution (logs) et les résultats de la simulation en temps réel.

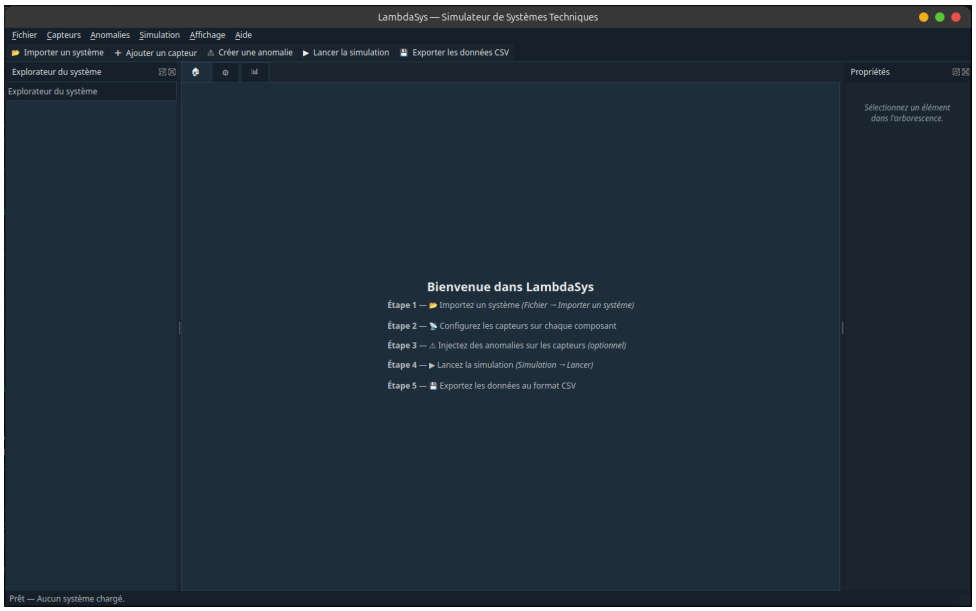


FIGURE 3.1 – Maquette de l'interface principale de LambdaSys

## 3.2 Modélisation UML

### 3.2.1 Axe Fonctionnel (Cas d'utilisation)

Le diagramme des cas d'utilisation met en évidence les interactions de l'utilisateur (Ingénieur) avec le système. Les cas d'utilisation principaux sont : "Importer un système", "Configurer les capteurs", "Lancer la simulation" et "Exporter les données (CSV)".

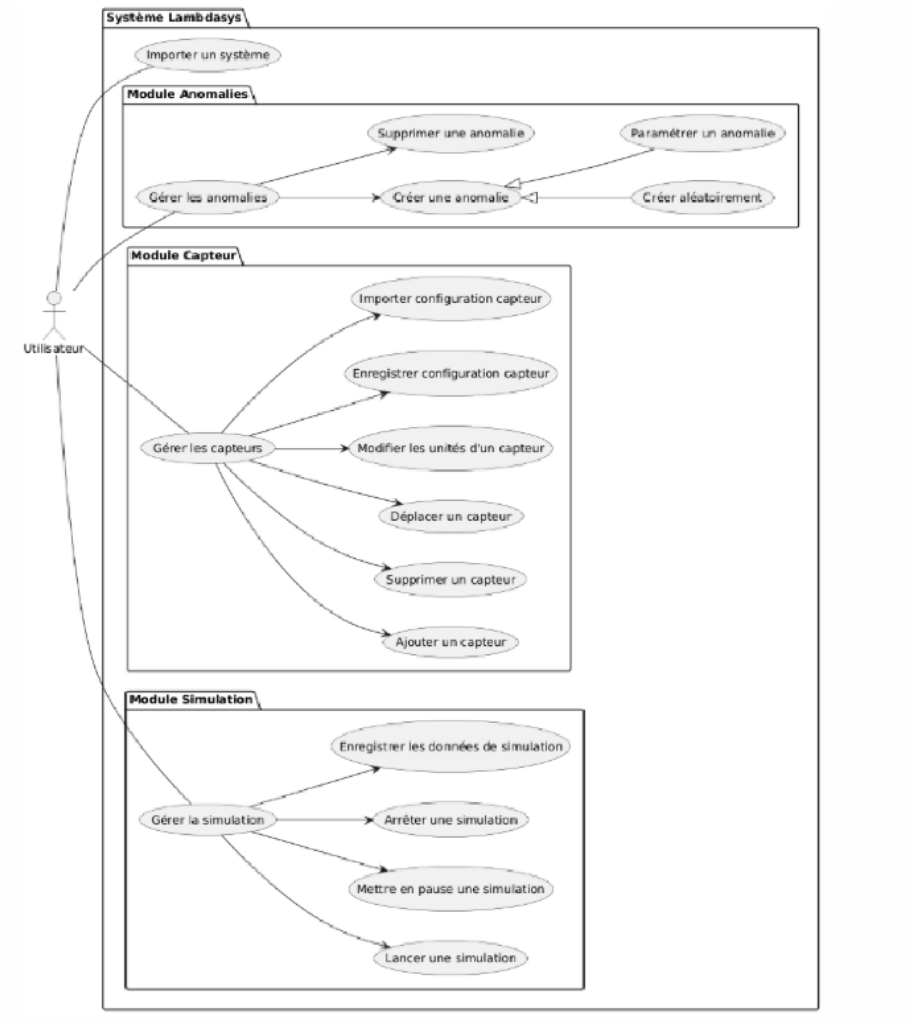


FIGURE 3.2 – Diagramme des cas d'usages

### 3.2.2 Axe Statique (Diagramme de Classes)

L'axe statique repose sur un motif d'architecture strict : le **MVC (Modèle-Vue-Contrôleur)**.

- **Le Modèle** est composé d'une classe abstraite mère générique (`Component`) dont héritent toutes les pièces du système technique importé.
- **La Vue** est gérée via la bibliothèque PyQt5 (`MainWindowGUI`), s'occupant exclusivement de l'affichage.
- **Le Contrôleur** (`MainWindowController`) orchestre les actions de l'utilisateur et interagit avec le moteur de simulation (`SimulationEngine`).

### 3.2.3 Axe Dynamique (Diagramme de Séquence)

Le diagramme de séquence illustre le scénario nominal de la gestion d'une simulation. Il détaille la boucle temporelle : pour chaque pas de temps, le contrôleur demande au modèle de calculer son nouvel état physique, puis récupère les valeurs des capteurs pour les stocker en mémoire avant de mettre à jour l'interface.

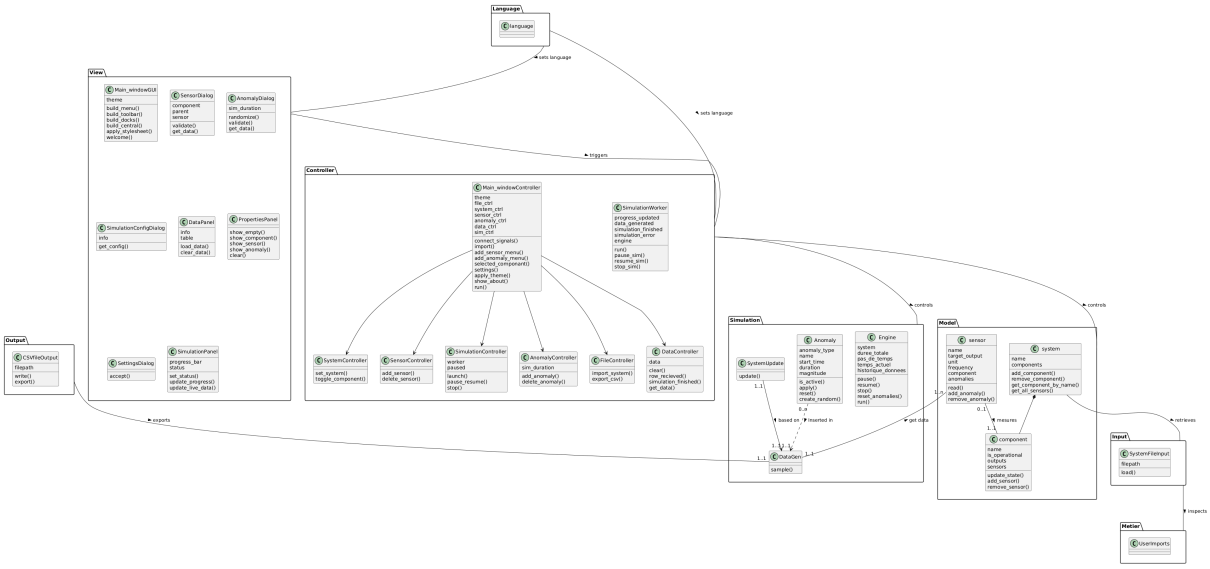


FIGURE 3.3 – Diagramme de classes détaillant l'architecture MVC

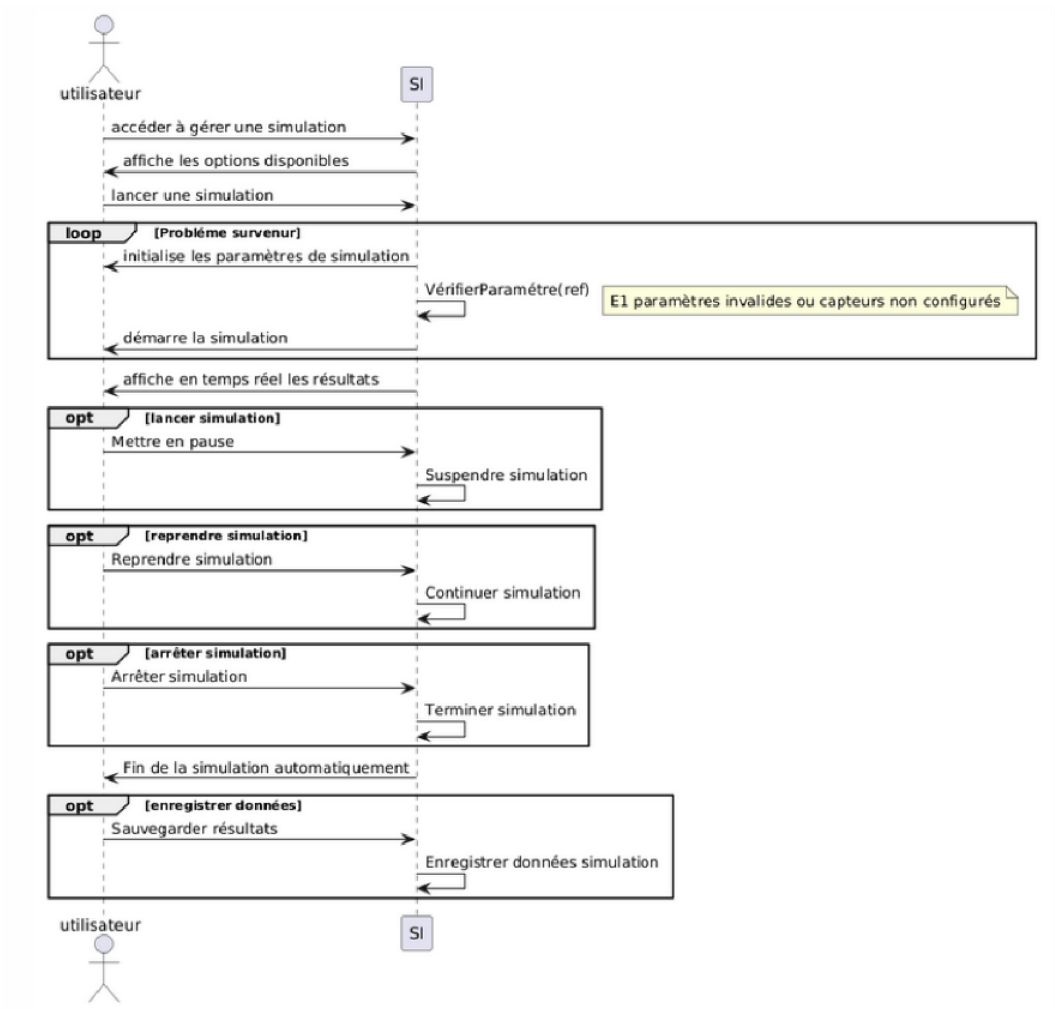


FIGURE 3.4 – Diagramme de séquence : Exécution d'une simulation

# 4. Implémentation et Bilan Technique

## 4.1 Choix techniques : Introspection et Généricité

Pour répondre à la contrainte stricte de concevoir un logiciel "générique", nous avons fait le choix d'utiliser l'introspection en Python (via les bibliothèques natives `importlib` et `inspect`).

Plutôt que de coder les composants techniques (pompe, valve, moteur) en dur dans le logiciel, LambdaSys charge dynamiquement des scripts `.py` externes fournis par l'utilisateur. Le logiciel inspecte le contenu du fichier à la volée, identifie les classes héritant de notre classe de base, et construit l'arborescence de l'interface graphique automatiquement. Cela rend l'outil adaptable à n'importe quel domaine d'ingénierie.

## 4.2 Problèmes rencontrés et solutions

Durant la phase de développement, plusieurs défis techniques se sont présentés :

- **Débordement de l'interface graphique :** Lors des premiers tests de simulation, l'affichage du dictionnaire de données dans un simple `QLabel` provoquait un étirement non désiré de la fenêtre. Ce problème a été résolu en utilisant un `QTextEdit` qui gère nativement les barres de défilement.
- **Environnement de développement Linux :** Des conflits de paquets (PEP 668) lors de l'installation de PyQt5 ont été contournés par la mise en place rigoureuse d'environnements virtuels (`venv`), garantissant ainsi la portabilité de notre code sur différentes machines.

# 5. Difficultées rencontrées

Au cours du projet, nous avons du faire face à différents problèmes. Ces problèmes allant du plus basique aux plus complexes nous ont permis d'apprendre à mieux structurer le projet et à éviter des incohérences qui auraient pu nous bloquer durant la conception du projet. Cela commence dès le depart lorsqu'il fallait réflechir à la façon dont on allait aborder le problème.

Une fois avoir bien identifier les differentes tâches à faire vient la partie UML qui n'a pas été simple mais qui nous a pas poser de grandes difficultés pour autant. L'axe statique a été le plus contraignant, notamment la partie sur le diagramme de classe qui s'avère particulièrement difficiles car il fallait très souvent revenir dessus lorsque l'implémentation en Python devenait fastidieux.

En ce qui concerne la partie Implementation, il s'agit de la partie la plus complexe pour plusieurs raisons. La première étant la strcuture du code car le fait d'avoir un fichier par classe rend certe le code très polyvalent mais devient très difficile à manipuler surtout lorsque l'on souhaite faire des modifications. Il était do,c important de bien séparer les différents fichier et surtout de bien les structurer.

Nous avons également rencontré des difficultés sur les anomalies et la simulation. Les anomalies étaient mal intégré au projet et pouvait faire parfois planter la simulation, un problème qui a été corrigé rapidement. La simulation a reçu énormement de modification, ce qui a nécéssité plusieurs de jour de travail pour avoir un résultat idéal.

Pour le reste du projet nous n'avons rencontré aucune difficulté significative.

# 6. Conclusion

Le projet LambdaSys a permis de concevoir avec succès un logiciel de simulation modulaire et générique. En respectant une méthodologie Agile rigoureuse et en s'appuyant sur une conception UML solide, nous avons pu implémenter une architecture MVC robuste en Python.

La première phase du cahier des charges est pleinement validée : l'utilisateur peut importer un modèle externe, configurer des capteurs, lancer une simulation temporelle et exporter les données nominales générées au format CSV. Ces jeux de données, garantis sans biais non maîtrisés, constituent désormais une base solide et exploitable pour la future étape du projet : l'entraînement et la validation de modèles d'Intelligence Artificielle pour la maintenance prédictive.